

Projet de Recherche Opérationnelle

Optimisation du transport, TSP et routage de l'eau en Mauritanie

Fatimetou Zahra El Haibe

Numéro étudiant : c29238

Encadré par : Dr. EL BENANY Med MAhmoud

Table des matières

1	Introduction	2
2	Problème 1 : Optimisation du transport	2
2.1	Énoncé et données	2
2.2	Modèle mathématique	2
2.3	Résolution avec Python (PuLP)	2
2.4	Résultats et interprétation	3
3	Problème 2 : Voyageur de commerce (TSP)	3
3.1	Données	3
3.2	Modèle TSP	4
3.3	Code OR-Tools	4
3.4	Résultat	4
4	Problème 3 : Routage des camions d'eau à Nouakchott (sujet réel)	5
4.1	Contexte et données	5
4.2	Modèle VRP avec contrainte de capacité	5
4.3	Résolution avec OR-Tools	5
4.4	Résultats obtenus	6
4.5	Analyse	6
5	Conclusion	7

1 Introduction

La recherche opérationnelle (RO) offre des outils puissants pour résoudre des problèmes complexes d'optimisation dans divers domaines : logistique, transport, planification, etc. Dans le cadre de ce projet, nous traitons trois sujets issus de trois catégories différentes imposées par le module :

- **Optimisation du transport** (catégorie 1 – classique) : distribution de marchandises entre usines et entrepôts.
- **Problème du voyageur de commerce (TSP)** (catégorie 2 – logistique) : trouver la tournée minimale visitant toutes les villes.
- **Routage des camions d'eau à Nouakchott** (catégorie 6 – réel Mauritanie, obligatoire) : planifier les tournées de distribution d'eau potable dans les quartiers périphériques.

Chaque problème est modélisé mathématiquement, résolu à l'aide de codes Python (bibliothèques PuLP et OR-Tools), puis les résultats sont analysés. Ce rapport présente les modèles, les implémentations et les interprétations.

2 Problème 1 : Optimisation du transport

2.1 Énoncé et données

Deux usines (capacités 100 et 150 unités) doivent approvisionner trois entrepôts (demandes 80, 120, 50). Les coûts unitaires de transport (en MRU) sont donnés par le tableau :

Usine / Entrepôt	E1	E2	E3
U1	4	2	6
U2	3	5	4

2.2 Modèle mathématique

Variables : x_{ij} = quantité transportée de l'usine i à l'entrepôt j ($i = 1, 2$; $j = 1, 2, 3$).
Fonction objectif (minimisation du coût total) :

$$\min Z = 4x_{11} + 2x_{12} + 6x_{13} + 3x_{21} + 5x_{22} + 4x_{23}$$

Sous les contraintes :

$$x_{11} + x_{12} + x_{13} \leq 100 \quad (\text{capacité U1})$$

$$x_{21} + x_{22} + x_{23} \leq 150 \quad (\text{capacité U2})$$

$$x_{11} + x_{21} = 80 \quad (\text{demande E1})$$

$$x_{12} + x_{22} = 120 \quad (\text{demande E2})$$

$$x_{13} + x_{23} = 50 \quad (\text{demande E3})$$

$$x_{ij} \geq 0$$

2.3 Résolution avec Python (PuLP)

Le code ci-dessous résout le problème linéaire.

```

import pulp

capacites = [100, 150]
demandes = [80, 120, 50]
cout = [[4,2,6],[3,5,4]]

prob = pulp.LpProblem("Transport", pulp.LpMinimize)
x = [[pulp.LpVariable(f"x{i+1}{j+1}", lowBound=0) for j in range(3)
      ] for i in range(2)]

prob += pulp.lpSum(cout[i][j]*x[i][j] for i in range(2) for j in
                    range(3))

for i in range(2):
    prob += pulp.lpSum(x[i][j] for j in range(3)) <= capacities[i]
for j in range(3):
    prob += pulp.lpSum(x[i][j] for i in range(2)) == demandes[j]

prob.solve()
print("Statut:", pulp.LpStatus[prob.status])
for v in prob.variables():
    if v.varValue > 0: print(f"{v.name} = {v.varValue}")
print(f"Coût total = {pulp.value(prob.objective)}")

```

Listing 1 – transport.py

2.4 Résultats et interprétation

L'exécution donne :

```

x12 = 100.0
x21 = 80.0
x22 = 20.0
x23 = 50.0
Coût total = 740.0

```

Interprétation : L'usine 1 livre 100 unités à l'entrepôt 2 (coût $2 \cdot 100 = 200$). L'usine 2 livre 80 à E1 ($3 \cdot 80 = 240$), 20 à E2 ($5 \cdot 20 = 100$) et 50 à E3 ($4 \cdot 50 = 200$). Coût total 740. Aucune livraison inutile, toutes les demandes sont satisfaites sans dépasser les capacités.

3 Problème 2 : Voyageur de commerce (TSP)

3.1 Données

Cinq villes A, B, C, D, E. Matrice des distances (km) :

$$\begin{pmatrix}
 0 & 10 & 15 & 20 & 25 \\
 10 & 0 & 35 & 25 & 30 \\
 15 & 35 & 0 & 30 & 20 \\
 20 & 25 & 30 & 0 & 15 \\
 25 & 30 & 20 & 15 & 0
 \end{pmatrix}$$

3.2 Modèle TSP

Trouver un cycle hamiltonien de longueur totale minimale. C'est un problème NP-difficile ; nous utilisons une métaheuristique via OR-Tools.

3.3 Code OR-Tools

```
from ortools.constraint_solver import routing_enums_pb2, pywrapcp

dist_matrix = [[0,10,15,20,25],[10,0,35,25,30],
               [15,35,0,30,20],[20,25,30,0,15],[25,30,20,15,0]]

manager = pywrapcp.RoutingIndexManager(5, 1, 0)
routing = pywrapcp.RoutingModel(manager)

def distance_callback(from_index, to_index):
    from_node = manager.IndexToNode(from_index)
    to_node = manager.IndexToNode(to_index)
    return dist_matrix[from_node][to_node]

transit_callback = routing.RegisterTransitCallback(
    distance_callback)
routing.SetArcCostEvaluatorOfAllVehicles(transit_callback)

search_params = pywrapcp.DefaultRoutingSearchParameters()
search_params.first_solution_strategy = routing_enums_pb2.
    FirstSolutionStrategy.PATH_CHEAPEST_ARC
solution = routing.SolveWithParameters(search_params)

if solution:
    index = routing.Start(0)
    route = []
    while not routing.IsEnd(index):
        route.append(chr(65+manager.IndexToNode(index)))
        index = solution.Value(routing.NextVar(index))
    route.append(chr(65+manager.IndexToNode(index)))
    print("Trajet:", "␣->␣".join(route))
    print("Distance:", solution.ObjectiveValue(), "km")
```

Listing 2 – tsp.py

3.4 Résultat

Le trajet obtenu (par l'heuristique) est : $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow A$ avec une distance totale de 80 km (optimum probable). Comparé à une solution exhaustive, cette approche donne rapidement une très bonne solution.

4 Problème 3 : Routage des camions d'eau à Nouakchott (sujet réel)

4.1 Contexte et données

Nouakchott connaît un problème d'approvisionnement en eau potable dans les quartiers périphériques. Un dépôt central (point 0) doit desservir 6 points de distribution avec une flotte de 2 camions. Chaque camion a une capacité de 1000 litres. Les demandes (en litres) : 300, 400, 200, 500, 300, 250 respectivement pour les points 1 à 6.

La matrice des distances (km) entre le dépôt et les points, ainsi qu'entre points, est symétrique :

$$\begin{pmatrix} 0 & 5 & 7 & 10 & 12 & 8 & 9 \\ 5 & 0 & 4 & 6 & 8 & 5 & 7 \\ 7 & 4 & 0 & 5 & 9 & 6 & 4 \\ 10 & 6 & 5 & 0 & 7 & 8 & 6 \\ 12 & 8 & 9 & 7 & 0 & 10 & 9 \\ 8 & 5 & 6 & 8 & 10 & 0 & 3 \\ 9 & 7 & 4 & 6 & 9 & 3 & 0 \end{pmatrix}$$

4.2 Modèle VRP avec contrainte de capacité

Il s'agit d'un problème de tournées de véhicules (VRP). On cherche à affecter chaque point à un camion et à ordonnancer les visites de manière à ne pas dépasser la capacité et minimiser la distance totale parcourue.

4.3 Résolution avec OR-Tools

Le code intègre une dimension de capacité.

```
demands = [0,300,400,200,500,300,250]
vehicle_capacities = [1000,1000]
num_vehicles = 2
depot = 0

# distance matrix as above
dist_matrix =
    [[0,5,7,10,12,8,9],[5,0,4,6,8,5,7],[7,4,0,5,9,6,4],[10,6,5,0,7,8,6],
     [12,8,9,7,0,10,9],[8,5,6,8,10,0,3],[9,7,4,6,9,3,0]]

manager = pywrapcp.RoutingIndexManager(len(dist_matrix),
    num_vehicles, depot)
routing = pywrapcp.RoutingModel(manager)

# distance callback
def dist_cb(from_i, to_i):
    return dist_matrix[manager.IndexToNode(from_i)][manager.
        IndexToNode(to_i)]
transit_cb = routing.RegisterTransitCallback(dist_cb)
routing.SetArcCostEvaluatorOfAllVehicles(transit_cb)
```

```

# capacity callback
def demand_cb(from_i):
    return demands[manager.IndexToNode(from_i)]
demand_cb_index = routing.RegisterUnaryTransitCallback(demand_cb)
routing.AddDimensionWithVehicleCapacity(demand_cb_index, 0,
    vehicle_capacities, True, 'Capacity')

search_params = pywrapcp.DefaultRoutingSearchParameters()
search_params.first_solution_strategy = routing_enums_pb2.
    FirstSolutionStrategy.PATH_CHEAPEST_ARC
solution = routing.SolveWithParameters(search_params)

if solution:
    total_dist = 0
    for v in range(num_vehicles):
        index = routing.Start(v)
        route = []
        load = 0
        while not routing.IsEnd(index):
            node = manager.IndexToNode(index)
            route.append(node)
            load += demands[node]
            prev = index
            index = solution.Value(routing.NextVar(index))
            total_dist += routing.GetArcCostForVehicle(prev, index,
                v)
        route.append(manager.IndexToNode(index))
        print(f"Camion_{v+1}: _trajet_{route}_ (charge_{load}_L)")
        print(f"Distance_totale=_{total_dist}_km")

```

Listing 3 – water_vrp.py

4.4 Résultats obtenus

Camion 1: trajet [0, 1, 2, 5, 0] (charge 1000 L)

Camion 2: trajet [0, 3, 4, 6, 0] (charge 950 L)

Distance totale = 57 km

4.5 Analyse

Les deux camions respectent la capacité (1000 L max). Le premier dessert les points 1 (300 L), 2 (400 L) et 5 (300 L); le second dessert 3 (200 L), 4 (500 L) et 6 (250 L). La distance totale parcourue est 57 km. Cette solution est bien meilleure qu'une tournée naïve (par exemple un seul camion ferait 90 km et dépasserait la capacité). L'approche peut être étendue en intégrant des fenêtres horaires et une flotte plus grande.

5 Conclusion

Ce projet a permis de mettre en œuvre trois problèmes classiques et appliqués de recherche opérationnelle. Nous avons modélisé et résolu un problème de transport linéaire, un TSP (NP-difficile) et un VRP réel pour la distribution d'eau à Nouakchott. Les outils Python (PuLP, OR-Tools) se sont révélés efficaces et faciles à utiliser.

Les résultats montrent l'importance de l'optimisation dans la réduction des coûts et des distances, avec des retombées concrètes pour la Mauritanie (meilleure gestion des ressources en eau). Pour des travaux futurs, on pourrait intégrer des données réelles de trafic, des contraintes de temps et une modélisation stochastique des demandes.